

Lab 07: Fabric IQ, Direct Lake & Semantic Governance

Objective: Build and tune a consolidated enterprise semantic model in Direct Lake mode with Fabric IQ layering, XMLA/TOM version governance, column statistics inspection, and VertiPaq performance optimization.

Case Study: Enterprise Sales Analytics & Semantic Model Optimization

Problem Statement

A large multinational retail organization operates across multiple product categories, customers, and sales regions. Its sales data is continuously generated and stored in an enterprise **Gold Lakehouse** in Microsoft Fabric.

The organization currently faces several challenges:

- Sales data is stored in large fact tables containing millions of transaction records.
- Different analytics teams create their own calculations and reports, resulting in inconsistent revenue and profitability definitions.
- Business users need a single trusted semantic model for enterprise reporting.
- Analysts require **Year-to-Date (YTD)** and **Year-over-Year (YoY)** analysis.
- The existing semantic model contains unnecessary columns and a monolithic fact-table structure.
- Large analytical queries can experience performance degradation.
- The organization needs governance metadata to track model versions, deployment dates, ownership, and changes.
- The data engineering and analytics teams want to use **Microsoft Fabric Direct Lake** so reports can query OneLake data without maintaining a traditional imported semantic model.

Business Requirement

The organization wants to build a **centralized enterprise sales analytics solution** in Microsoft Fabric that provides:

1. A trusted **Direct Lake semantic model** over Gold Lakehouse data.
2. A proper **star schema** connecting sales, customer, product, and date dimensions.
3. Standardized enterprise measures such as:
 - Total Revenue

- *Total Cost*
 - *Total Profit*
 - *Total Quantity*
 - *Transaction Count*
4. *Reusable time-intelligence calculations:*
 - *YTD*
 - *YoY Growth*
 5. *A governed semantic model with version and ownership metadata.*
 6. *Performance analysis using query execution and storage-level metrics.*
 7. *Optimization of unnecessary columns and the underlying Delta table layout.*
 8. *A business-facing report that allows management to analyze revenue by **Product Category and Calendar Year**.*

STEP 00: Prerequisites & Environment Setup (Not Counted)

- **Target Workspace:** Open your assigned Microsoft Fabric workspace with active Fabric capacity enabled.
- **Tooling Installation:** Ensure local installations of Power BI Desktop (Fabric mode), Tabular Editor 3, and DAX Studio are available on your workstation.
- **Permissions & Connectivity:** Verify that XMLA Read/Write is enabled at the tenant/capacity level. Copy the Workspace Connection URL via Workspace Settings > Premium > Workspace Connection.
- **Instructor-Provided Artifacts:** Confirm the instructor-provided Lakehouse containing the consolidated Gold delta tables (Fact_Sales, Dim_Customer, Dim_Date, Dim_Product) is online and accessible.

PHASE 2 — Create the Lakehouse

You are already inside the `volkswagen_data` workspace.

STEP 0.4 — Create Lakehouse

1. Go to your workspace

You should be on:

`volkswagen_data`

2. Click

+ New item

3. Search/select

Lakehouse

4. Enter exactly:

Lab07_Gold_Lakehouse

5. Click Create

Wait for Fabric to finish creating it.

STEP 0.5 — Open the Lakehouse

After creation, you should see:

Lab07_Gold_Lakehouse

Click it.

You should see something similar to:

Lab07_Gold_Lakehouse

Explorer

|

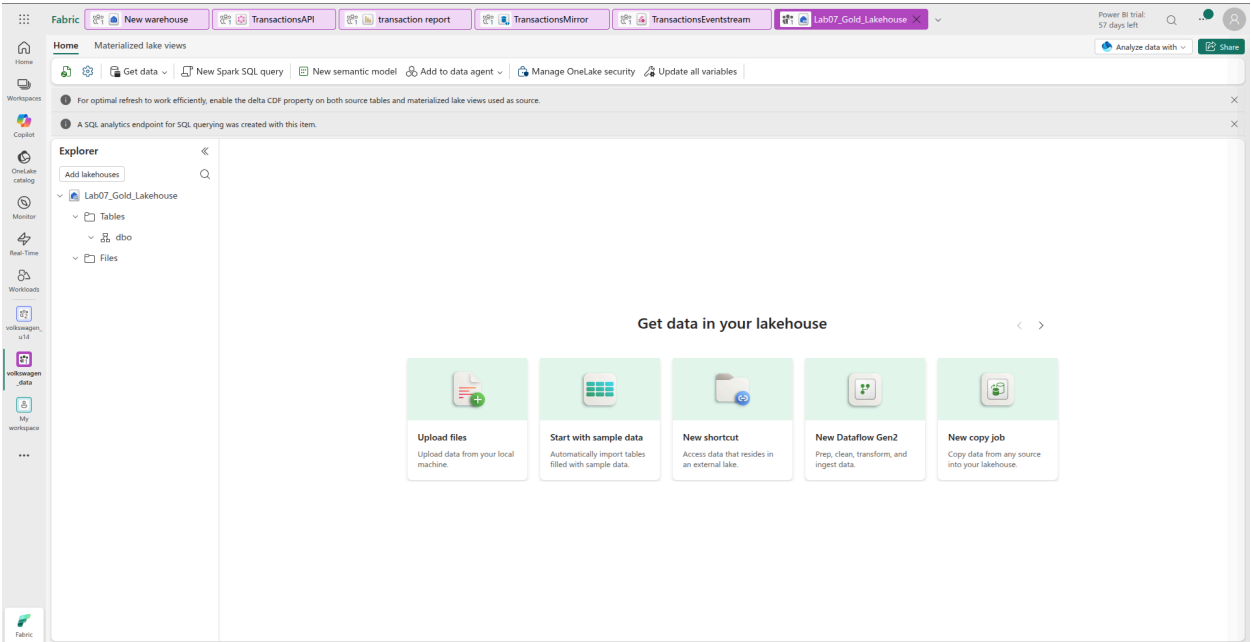
| — Tables

|

| — Files

Initially, Tables should be empty.

That's completely fine.



TEP 0.5 — Don't create tables manually

On your current screen, **do not click**:

- Upload files
- Start with sample data
- New shortcut
- New Dataflow

Instead, create a Notebook from the workspace.

1. Go back to `volkswagen_data`

Click the `volkswagen_data` **workspace** on the left sidebar.

2. Click

+ New item

3. Select

Notebook

4. Name it

Lab07_Data_Generator

Then click **Create**.

STEP 0.5.1 — Attach your Lakehouse

When the notebook opens, look at the **left Explorer pane**.

You should find something like:

Add data items

There should be an option:

Add lakehouse

Click it.

Then select:

Lab07_Gold_Lakehouse

and click **Add**.

After that, the notebook should show:

Lakehouses

└─ Lab07_Gold_Lakehouse

1. Create Dim_Customer

In the notebook, create a new code cell and paste:

```
from pyspark.sql import functions as F

customer_count = 1000

customers = (
    spark.range(1, customer_count + 1)
    .withColumnRenamed("id", "CustomerKey")

    .withColumn(
        "CustomerName",
        F.concat(
            F.lit("Customer "),
            F.col("CustomerKey").cast("string")
        )
    )

    .withColumn(
        "City",
        F.when((F.col("CustomerKey") % 8) == 1, "Pune")
        .when((F.col("CustomerKey") % 8) == 2, "Mumbai")
        .when((F.col("CustomerKey") % 8) == 3, "Delhi")
        .when((F.col("CustomerKey") % 8) == 4, "Bengaluru")
        .when((F.col("CustomerKey") % 8) == 5, "Hyderabad")
        .when((F.col("CustomerKey") % 8) == 6, "Chennai")
        .when((F.col("CustomerKey") % 8) == 7, "Kolkata")
        .otherwise("Ahmedabad")
    )

    .withColumn(
        "State",
        F.when(
            F.col("City").isin("Pune", "Mumbai"),
            "Maharashtra"
        )
    )
)
```

```

.when(
F.col("City") == "Delhi",
"Delhi"
)
.when(
F.col("City") == "Bengaluru",
"Karnataka"
)
.when(
F.col("City") == "Hyderabad",
"Telangana"
)
.when(
F.col("City") == "Chennai",
"Tamil Nadu"
)
.when(
F.col("City") == "Kolkata",
"West Bengal"
)
.otherwise("Gujarat")
)

.withColumn(
"Segment",
F.when((F.col("CustomerKey") % 3) == 1, "Consumer")
.when((F.col("CustomerKey") % 3) == 2, "Corporate")
.otherwise("Small Business")
)
)

customers.write \
.mode("overwrite") \
.format("delta") \
.saveAsTable("Dim_Customer")

```

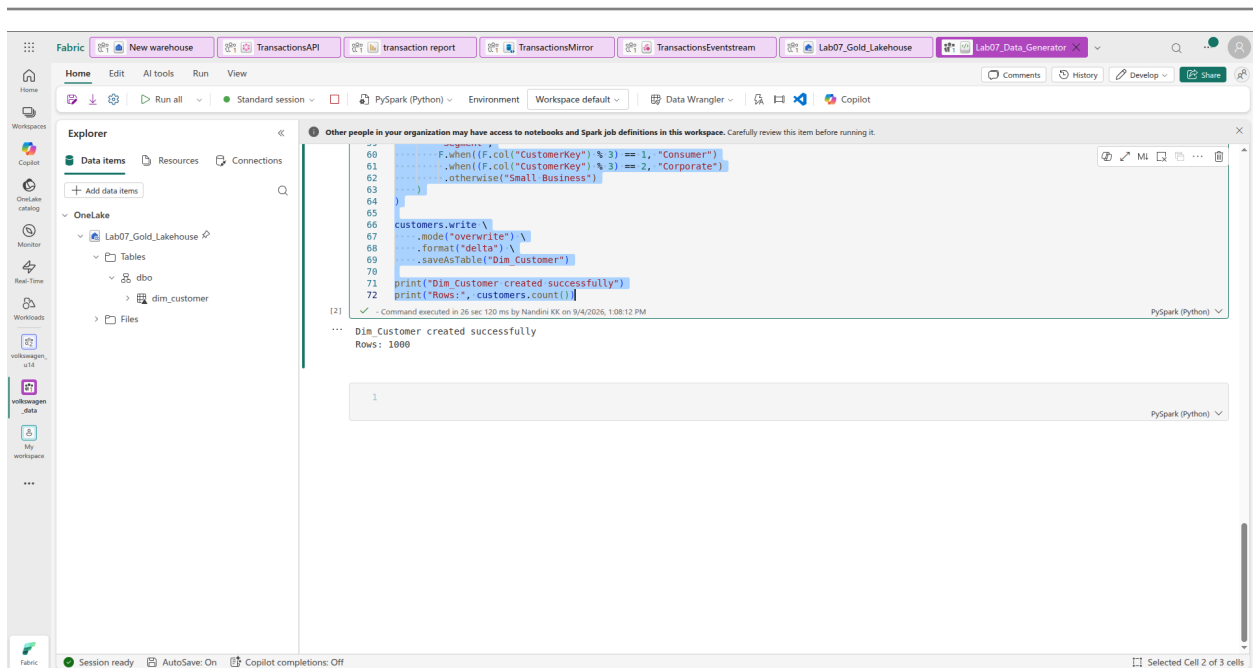
```
print("Dim_Customer created successfully")
print("Rows:", customers.count())
```

Run it with **Shift + Enter**.

Expected

Dim_Customer created successfully

Rows: 1000



2. Create Dim_Product

Create another cell:

```
from pyspark.sql import functions as F
```

```
# Create 100 products: ProductKey 101-200
```

```
products = (
    spark.range(101, 201)
    .withColumnRenamed("id", "ProductKey")
```

```
    .withColumn(
        "ProductName",
        F.concat(
```

```

F.lit("Product "),
F.col("ProductKey").cast("string")
)
)

.withColumn(
  "ProductCategory",
  F.when((F.col("ProductKey") % 3) == 0, "Electronics")
  .when((F.col("ProductKey") % 3) == 1, "Furniture")
  .otherwise("Office Supplies")
)

.withColumn(
  "ProductSubcategory",
  F.when(
    F.col("ProductCategory") == "Electronics",
    F.when(
      (F.col("ProductKey") % 2) == 0,
      "Computers"
    ).otherwise("Accessories")
  )
  .when(
    F.col("ProductCategory") == "Furniture",
    F.when(
      (F.col("ProductKey") % 2) == 0,
      "Chairs"
    ).otherwise("Tables")
  )
  .otherwise(
    F.when(
      (F.col("ProductKey") % 2) == 0,
      "Stationery"
    ).otherwise("Paper")
  )
)
)

```



```

.withColumn(
  "UnitPrice",
  F.round(
    F.lit(500) +
    ((F.col("ProductKey") * 137) % 74500),
    2
  )
)

# Write as Delta table
products.write \
  .mode("overwrite") \
  .format("delta") \
  .saveAsTable("Dim_Product")

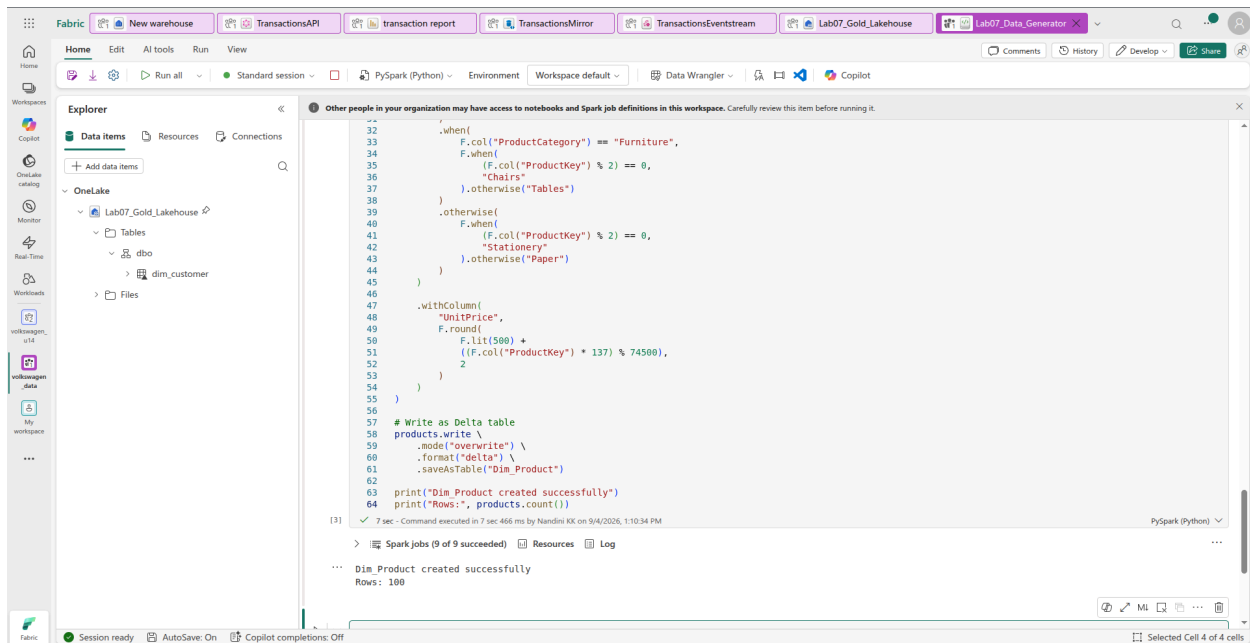
print("Dim_Product created successfully")
print("Rows:", products.count())

```

Expected:

Dim_Product created successfully

Rows: 100



3. Create Dim_Date

New cell:

```
from pyspark.sql import functions as F
```

```
# Create dates from 2025-01-01 to 2026-12-31
```

```
dates = (
    spark.sql("""
    SELECT explode(
    sequence(
    to_date('2025-01-01'),
    to_date('2026-12-31'),
    interval 1 day
    )
    ) AS Date
    """)
    .withColumn(
    "DateKey",
    F.date_format("Date", "yyyyMMdd").cast("int")
    )
    .withColumn(
    "CalendarYear",
```

```

F.year("Date")
)
.withColumn(
  "MonthNumber",
  F.month("Date")
)
.withColumn(
  "MonthName",
  F.date_format("Date", "MMMM")
)
.withColumn(
  "Quarter",
  F.concat(F.lit("Q"), F.quarter("Date"))
)
.select(
  "DateKey",
  "Date",
  "CalendarYear",
  "MonthNumber",
  "MonthName",
  "Quarter"
)
)

# Save as Delta table
dates.write \
  .mode("overwrite") \
  .format("delta") \
  .saveAsTable("Dim_Date")

print("Dim_Date created successfully")
print("Rows:", dates.count())

```

Run it.

Expected:

Dim_Date created successfully

Rows: 730

4. Create Fact_Sales

This is the main table and will contain **500,000 rows**.

Create a new cell:

```
from pyspark.sql import functions as F
```

```
# Number of sales records
```

```
sales_count = 500000
```

```
sales = (  
    spark.range(1, sales_count + 1)  
    .withColumnRenamed("id", "SalesKey")
```

```
# Transaction ID
```

```
.withColumn(  
    "TransactionKey",  
    (F.lit(1000000) + ((F.col("SalesKey") - 1) / 3).cast("long"))  
)
```

```
# 1-3 lines per transaction
```

```
.withColumn(  
    "TransactionID",  
    (F.lit(1000000) + ((F.col("SalesKey") - 1) / 3).cast("long"))
```

```

    "SalesOrderLineNumber",
    (((F.col("SalesKey") - 1) % 3) + 1).cast("int")
)

# Date distributed across 2025 and 2026
.withColumn(
    "Date",
    F.expr(
        "date_add("
        "to_date('2025-01-01'), "
        "cast((SalesKey - 1) % 730 as int)"
        ")"
    )
)

# YYYYMMDD integer key
.withColumn(
    "DateKey",
    F.date_format("Date", "yyyyMMdd").cast("int")
)

# CustomerKey: 1-1000
.withColumn(
    "CustomerKey",
    (((F.col("SalesKey") - 1) % 1000) + 1).cast("long")
)

# ProductKey: 101-200
.withColumn(
    "ProductKey",
    (((F.col("SalesKey") - 1) % 100) + 101).cast("long")
)

# Quantity: 1-10
.withColumn(
    "Quantity",
    (((F.col("SalesKey") - 1) % 10) + 1).cast("int")
)

```

```

)

# Revenue
.withColumn(
  "Revenue",
  F.round(
    F.col("Quantity") *
    (
      F.lit(500) +
      ((F.col("ProductKey") * 137) % 74500)
    ),
    2
  )
)

# Cost = 75% of revenue
.withColumn(
  "Cost",
  F.round(F.col("Revenue") * F.lit(0.75), 2)
)

.select(
  "SalesKey",
  "TransactionKey",
  "SalesOrderLineNumber",
  "DateKey",
  "CustomerKey",
  "ProductKey",
  "Quantity",
  "Revenue",
  "Cost"
)

# Write to Lakehouse as Delta
sales.write \
.mode("overwrite") \

```

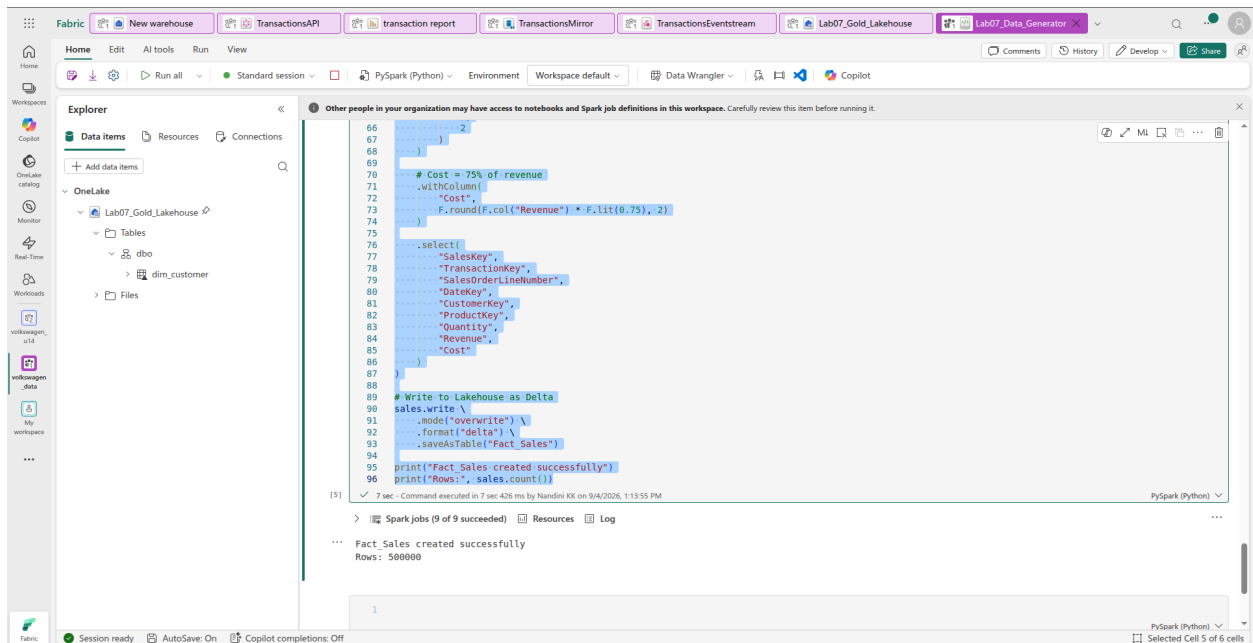
```
.format("delta") \
.saveAsTable("Fact_Sales")

print("Fact_Sales created successfully")
print("Rows:", sales.count())
```

Expected:

Fact_Sales created successfully

Rows: 500000



5. Verify all four tables

Finally, create one more cell:

```
tables = [
    "Dim_Customer",
    "Dim_Product",
    "Dim_Date",
    "Fact_Sales"
]
```

for table in tables:

```
count = spark.table(table).count()

print(f"{table}: {count:,} rows")
```

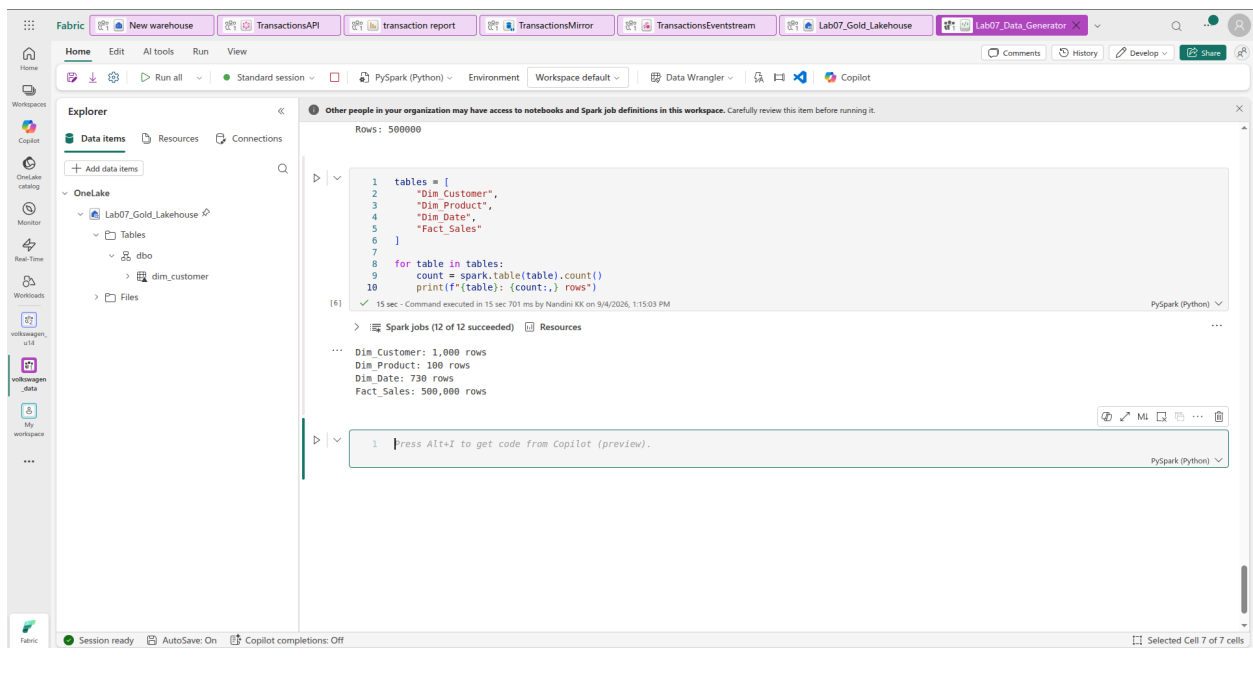
You should get:

Dim_Customer: 1,000 rows

Dim_Product: 100 rows

Dim_Date: 730 rows

Fact_Sales: 500,000 rows



6. Check the Lakehouse

Go back to:

Lab07_Gold_Lakehouse

Refresh the **Tables** section.

You should now have:

Tables

└─ Dim_Customer

└─ Dim_Date

└─ Dim_Product

└─ Fact_Sales

The screenshot shows the Microsoft Fabric interface with a table view of 'dim_customer'. The table has 5 columns: CustomerKey, CustomerName, City, State, and Segment. It displays 22 rows of customer data. The interface includes a sidebar with 'Explorer' and 'Lab07_Gold_Lakehouse' selected, and a top bar with various workspace tabs.

CustomerKey	CustomerName	City	State	Segment
1	Customer 1	Pune	Maharashtra	Consumer
2	Customer 2	Mumbai	Maharashtra	Corporate
3	Customer 3	Delhi	Delhi	Small Business
4	Customer 4	Bengaluru	Karnataka	Consumer
5	Customer 5	Hyderabad	Telangana	Corporate
6	Customer 6	Chennai	Tamil Nadu	Small Business
7	Customer 7	Kolkata	West Bengal	Consumer
8	Customer 8	Ahmedabad	Gujarat	Corporate
9	Customer 9	Pune	Maharashtra	Small Business
10	Customer 10	Mumbai	Maharashtra	Consumer
11	Customer 11	Delhi	Delhi	Corporate
12	Customer 12	Bengaluru	Karnataka	Small Business
13	Customer 13	Hyderabad	Telangana	Consumer
14	Customer 14	Chennai	Tamil Nadu	Corporate
15	Customer 15	Kolkata	West Bengal	Small Business
16	Customer 16	Ahmedabad	Gujarat	Consumer
17	Customer 17	Pune	Maharashtra	Corporate
18	Customer 18	Mumbai	Maharashtra	Small Business
19	Customer 19	Delhi	Delhi	Consumer
20	Customer 20	Bengaluru	Karnataka	Corporate
21	Customer 21	Hyderabad	Telangana	Small Business
22	Customer 22	Chennai	Tamil Nadu	Consumer

STEP 1 — Create the Direct Lake Semantic Model

The lab requires the consolidated model to contain:

- Fact_Sales
- Dim_Customer
- Dim_Product
- Dim_Date

and ultimately operate in **Direct Lake** mode.

1. Open your workspace

In Microsoft Fabric, open:

volswagen_data

Then open:

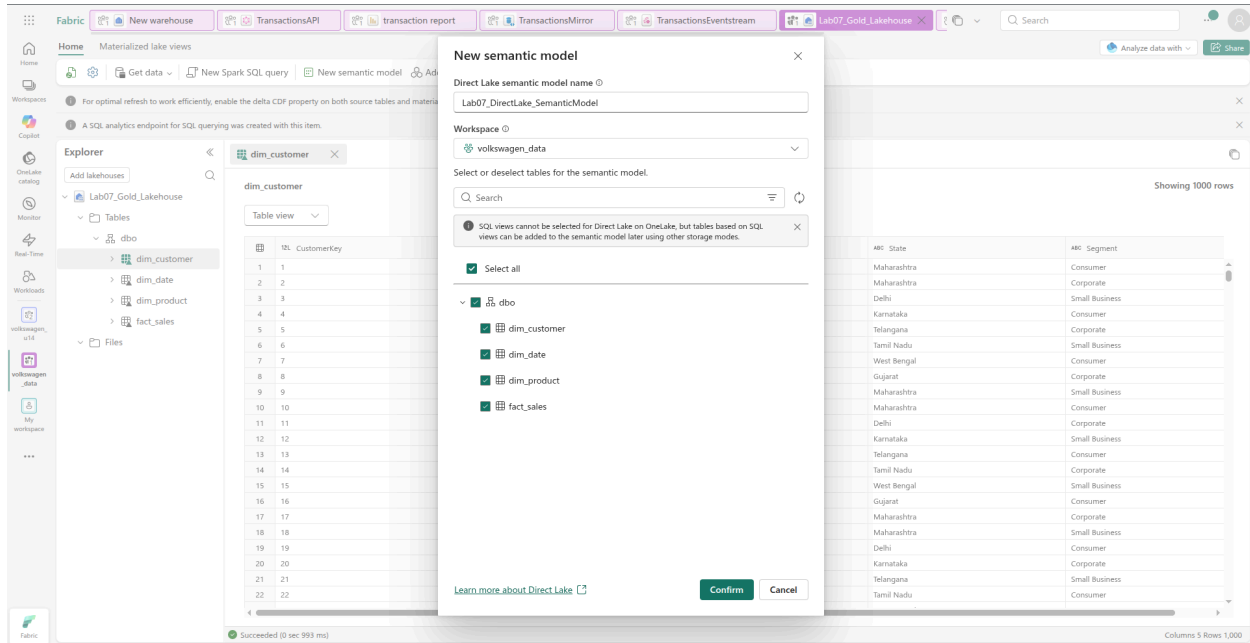
Lab07_Gold_Lakehouse

2. Click New semantic model

At the top of the Lakehouse page, click:

New semantic model

You should get a screen asking you to select tables.



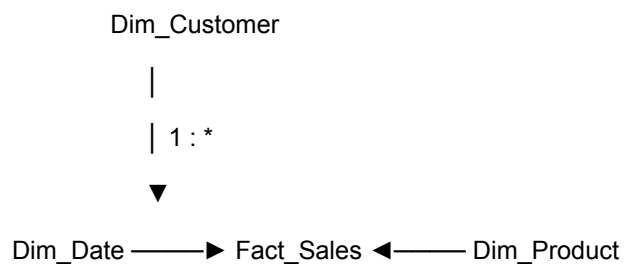
3. Select ONLY these four tables

Tick:

- ☒ Dim_Customer
- ☒ Dim_Date
- ☒ Dim_Product
- ☒ Fact_Sales

Do **not** select any of the unrelated tables/assets in your workspace.

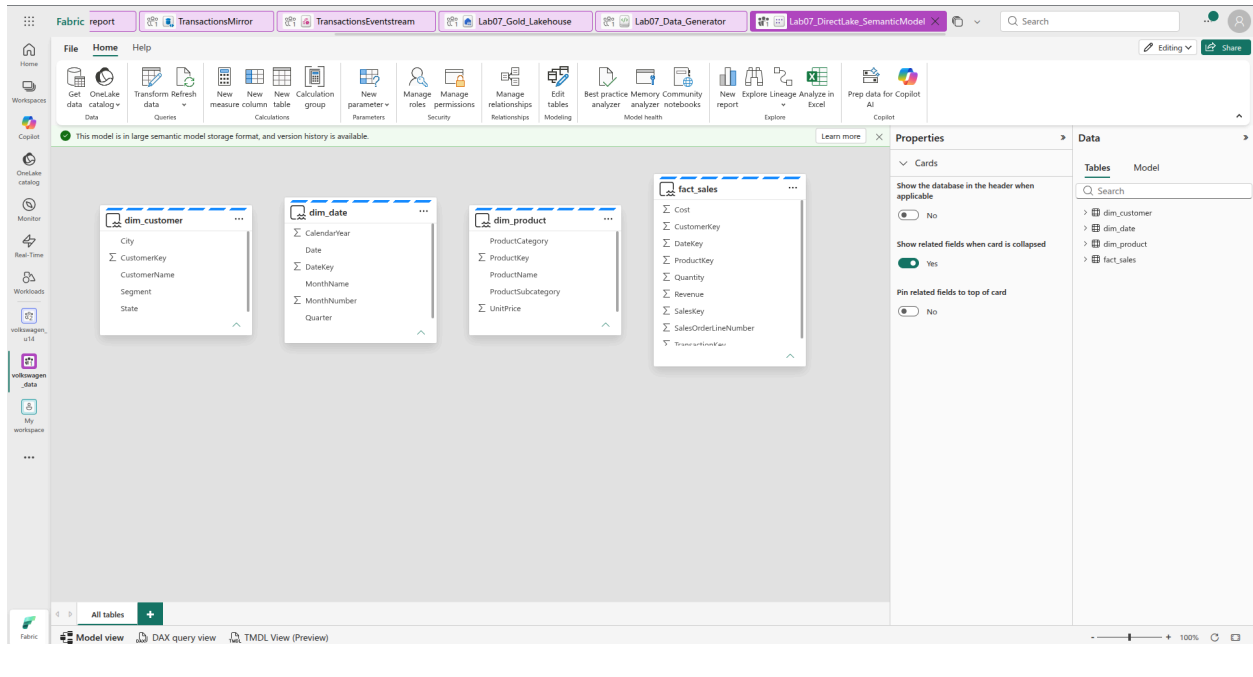
The resulting model should represent this star schema:



1 : *

1 : *

This is consistent with the lab's required modelling fundamentals.



STEP 4 — Create relationships

Click **Manage relationships** in the top toolbar.

You should get a relationships window.

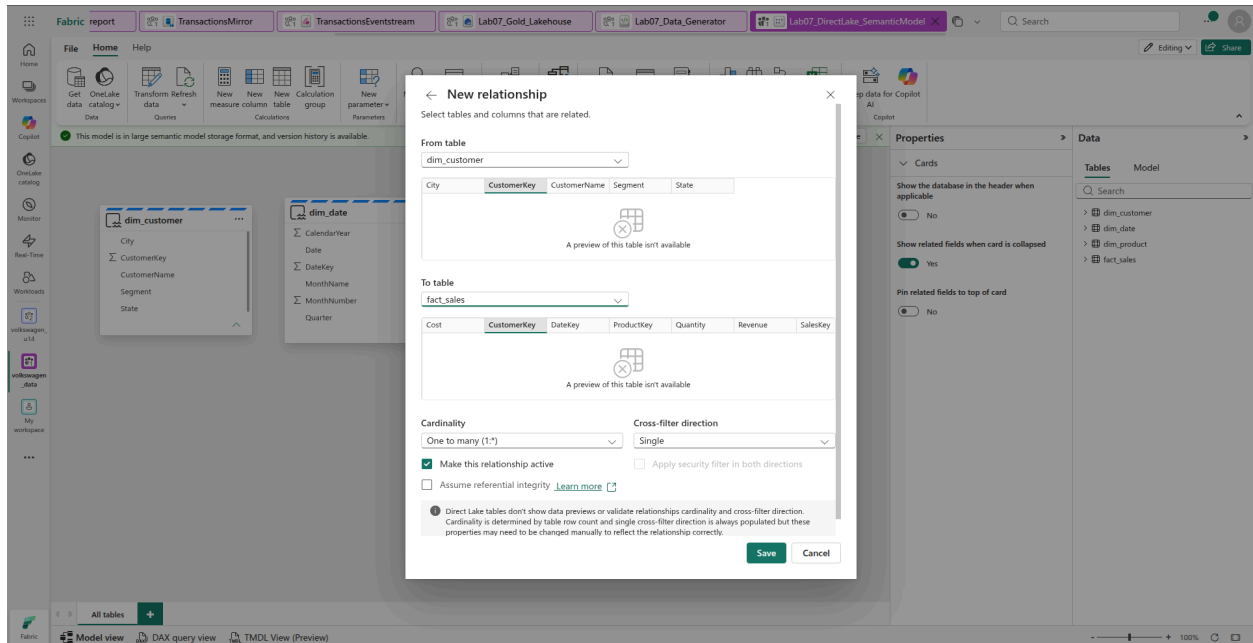
Relationship 1 — Customer

Click **New relationship**.

Set:

Setting	Value
From table	dim_customer
From column	CustomerKey
To table	fact_sales
To column	CustomerKey

Setting	Value
Cardinality	One to many (1:*)
Cross-filter direction	Single
Make this relationship active	☒
Then click Save / OK .	



Relationship 2 — Product

Create another relationship:

`dim_product[ProductKey]`

1
|
|
*

`fact_sales[ProductKey]`

Settings:

From table: `dim_product`

From column: `ProductKey`

To table: fact_sales

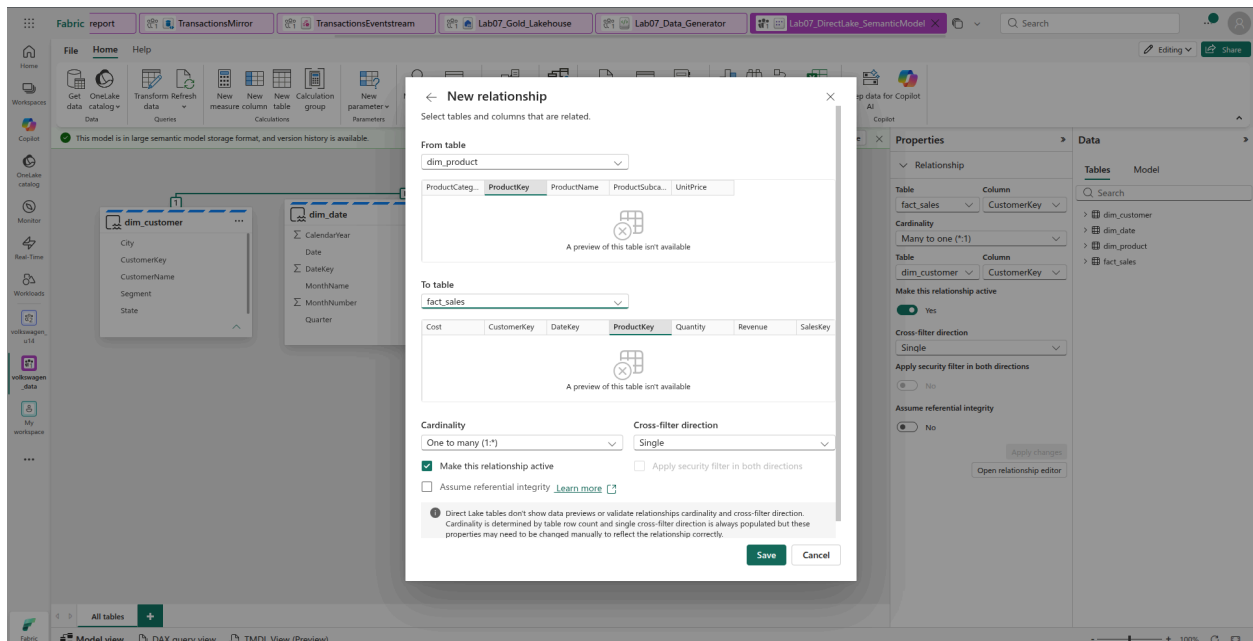
To column: ProductKey

Cardinality: One to many (1:*)

Cross-filter: Single

Active: Yes

Save it.



Relationship 3 — Date

Create the third:

dim_date[DateKey]

1

|

|

*

fact_sales[DateKey]

Settings:

From table: dim_date

From column: DateKey

To table: fact_sales

To column: DateKey

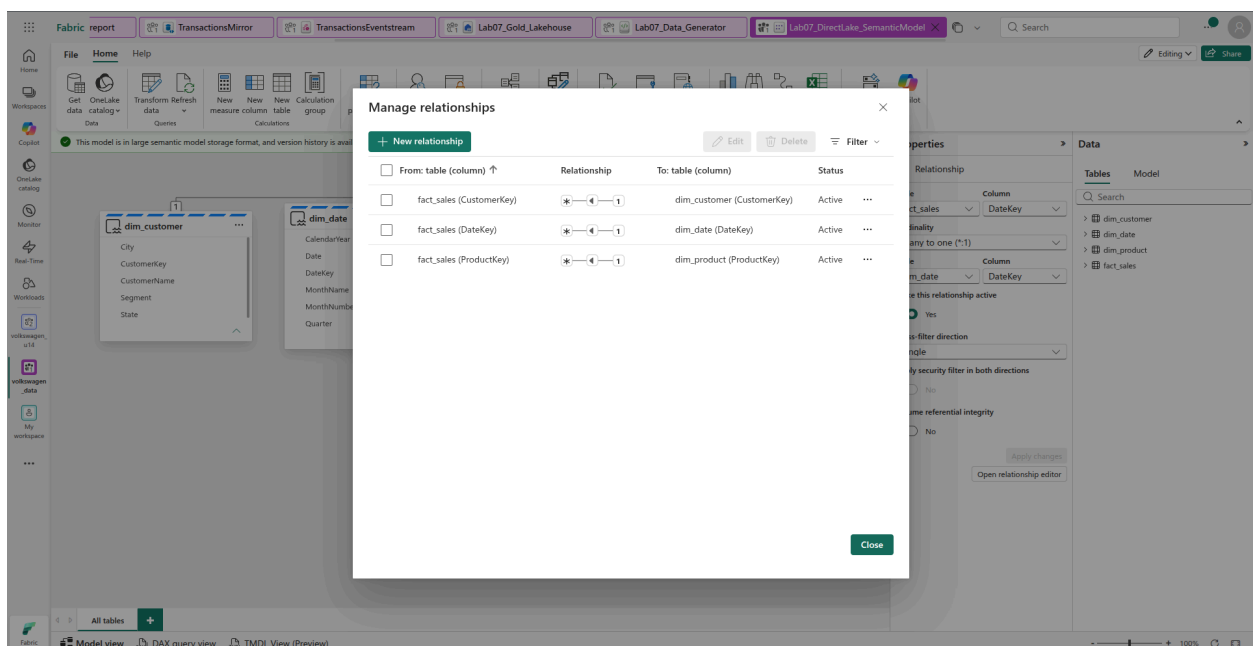
Cardinality: One to many (1:*)

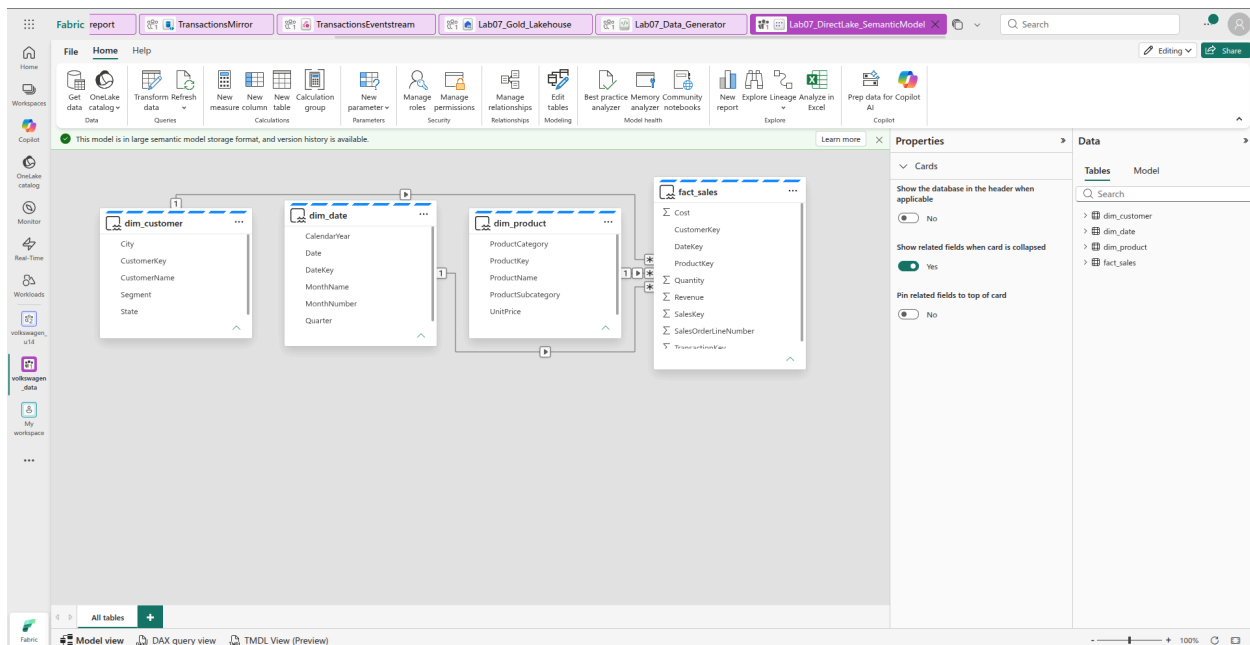
Cross-filter: Single

Active: Yes

Save it.

These three relationships give us the required star schema: the dimension tables on the **1-side** and fact_sales on the **many-side**.





STEP 5 — Create the required DAX measures

The lab requires measures and later uses [Total Revenue] for benchmarking and validation.

You are currently in **Model View**.

1. Create Total Revenue

At the top toolbar, click:

New measure

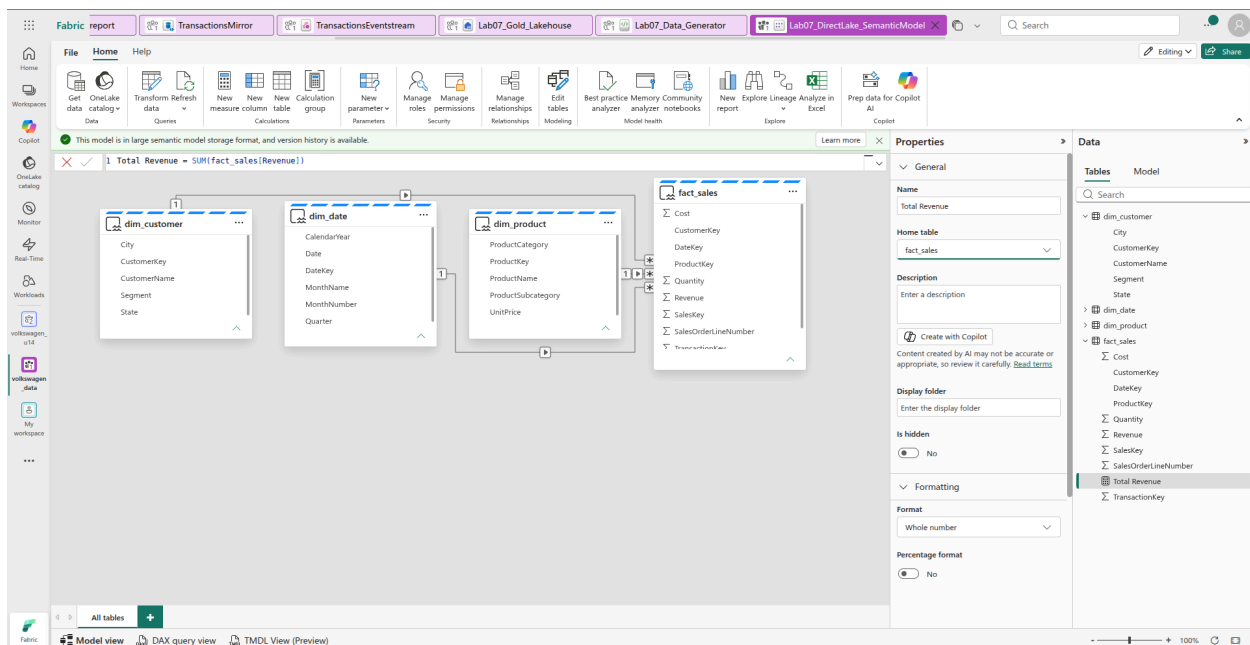
Select fact_sales first if Fabric asks which table should contain the measure.

Enter:

Total Revenue = SUM(fact_sales[Revenue])

Press **Enter**.

Green tick will be seen on top left after completion.

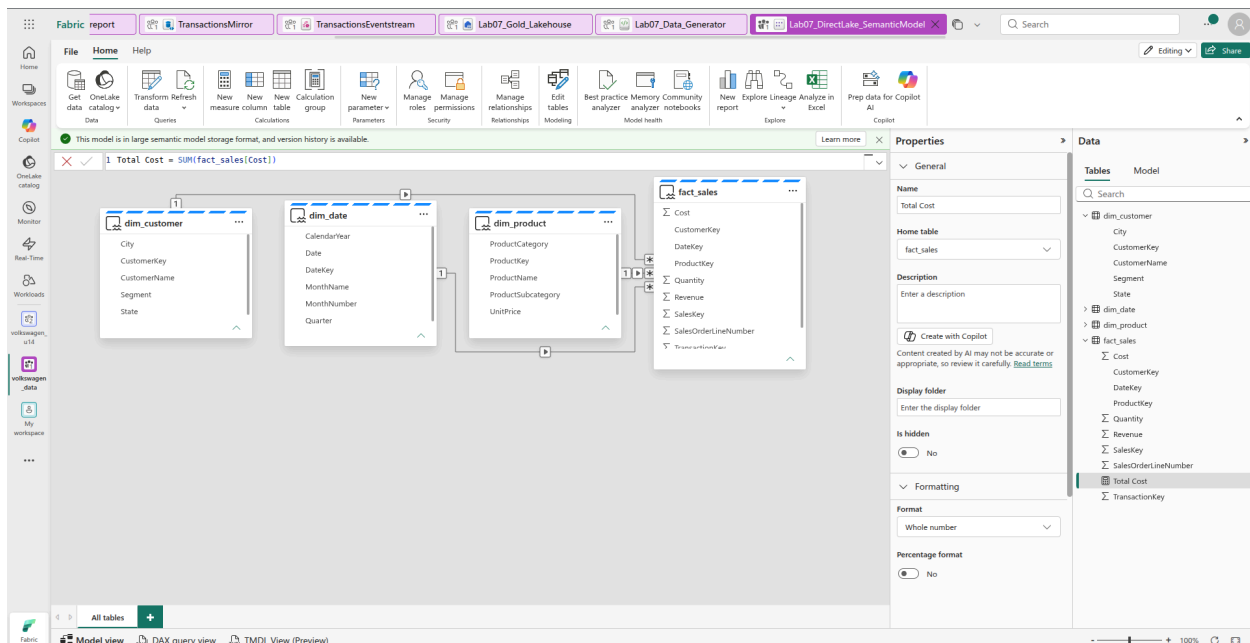


2. Create Total Cost

Click **New measure** again:

Total Cost = SUM(fact_sales[Cost])

Press **Enter**.



3. Create Total Profit separately

Again:

fact_sales → **New measure**

Enter only:

Total Profit = [Total Revenue] - [Total Cost]

STEP 6 — Verify the measures

In the right-hand **Data** pane, expand:

fact_sales

You should eventually see:

Σ Cost

Σ CustomerKey

Σ DateKey

Σ ProductKey

Σ Quantity

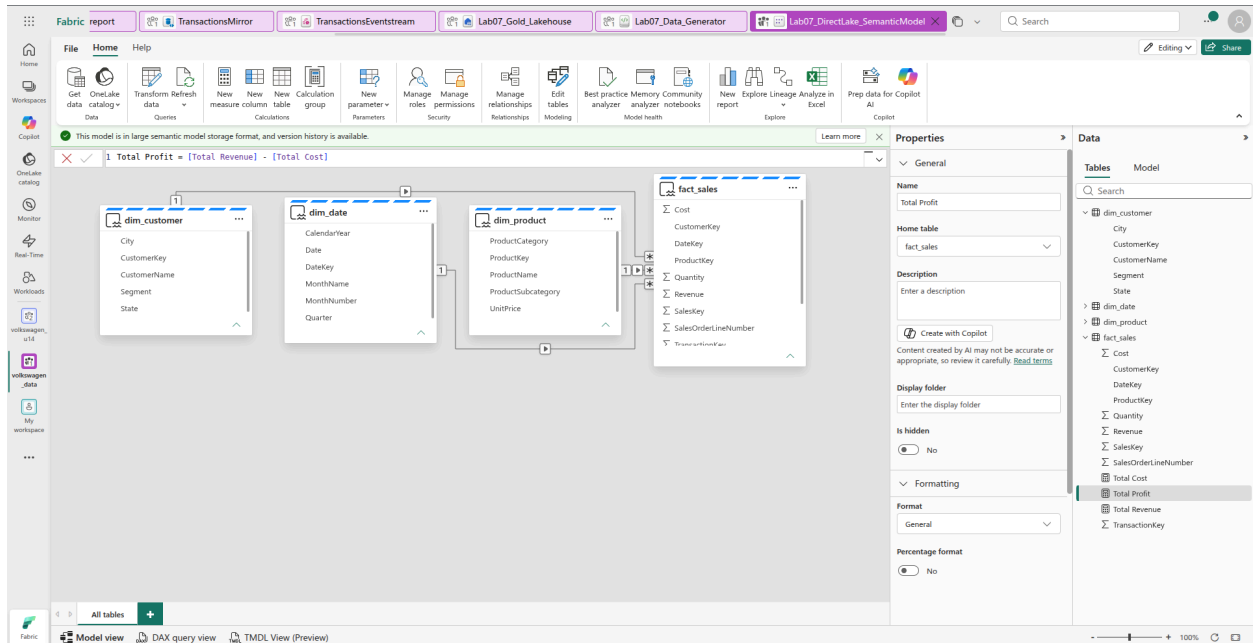
Σ Revenue

Σ SalesKey

...

Σ Total Revenue

Σ Total Cost



STEP 7 — Create the TimeIntelligence calculation group

This is required by the lab. The lab specifically expects a calculation group named **TimeIntelligence** with **YTD** and **YoY Growth** calculations.

Important

Don't create a normal table called TimeIntelligence.

We need a **Calculation Group**.

Do this:

At the top toolbar, you have:

New measure | **New column** | **New table** | **Calculation group**

Click:

Calculation group

Yes — this screenshot shows that **the Calculation Group has been created successfully.** 🍌

Right now Fabric has automatically created:

Calculation group

└─ Calculation group column

└─ Calculation items (1)

└─ Calculation item

We now need to rename these and create the two required calculation items.

Step 1 — Rename the Calculation Group

On the **right side**, under **Semantic model** → **Calculation groups**, click:

Calculation group

⚠ Do **not** click Calculation item.

Then look at the **Properties** → **Name** box.

Change:

Calculation group

to:

TimeIntelligence

Press **Enter**.

Step 2 — Rename the existing Calculation Item

Under:

Calculation items (1)

click:

Calculation item

The Properties pane should show:

Name: Calculation item

Change the name to:

YTD

Press **Enter**.

Step 3 — Enter the YTD formula

With **YTD** selected, click inside the formula bar at the top.

It currently says something like:

Calculation item = SELECTEDMEASURE()

Replace the **entire formula** with:

```
CALCULATE(  
    SELECTEDMEASURE(),  
    DATESYTD('dim_date'[Date])  
)
```

Then click the ✓ at the left of the formula bar.

Your structure should now look like:

Calculation groups

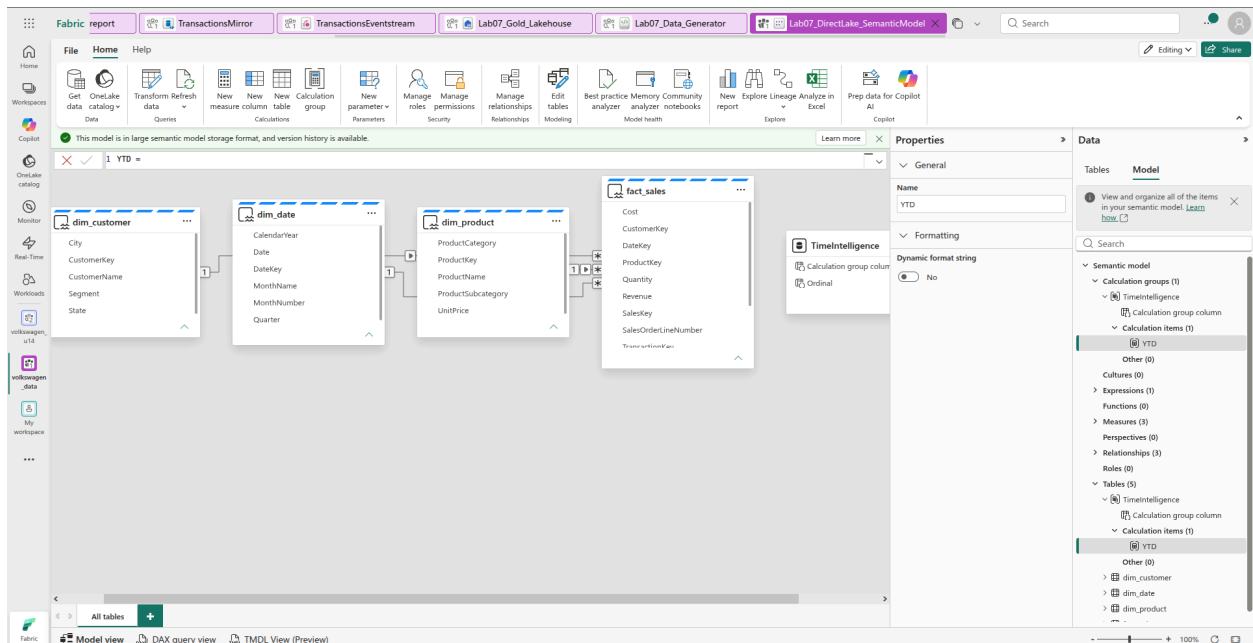
└─ TimeIntelligence

└─ Calculation group column

└─ Calculation items (1)

└─ YTD

The lab requires the TimeIntelligence calculation group with **YTD** and **YoY Growth** items.



STEP 4 — Create YoY Growth

1. Select TimeIntelligence

On the right **Model** pane, expand:

Semantic model

└─ Calculation groups

 └─ TimeIntelligence

Then find:

Calculation items (1)

└─ YTD

2. Create another calculation item

Right-click Calculation items under TimeIntelligence.

Choose:

New calculation item

Fabric should create another item, probably named:

Calculation item 2

3. Rename it

Select the new calculation item.

In **Properties** → **Name**, change it to:

YoY Growth

4. Enter the YoY formula

With YoY Growth selected, replace the formula at the top with:

VAR CurrentValue =

 SELECTEDMEASURE()

VAR PreviousYearValue =

 CALCULATE(

 SELECTEDMEASURE(),

 SAMEPERIODLASTYEAR('dim_date'[Date])

)

RETURN

DIVIDE(

CurrentValue - PreviousYearValue,

PreviousYearValue

)

Then click the ✓ at the top-left.

5. Verify

Your model should now show:

Calculation groups

└─ TimeIntelligence

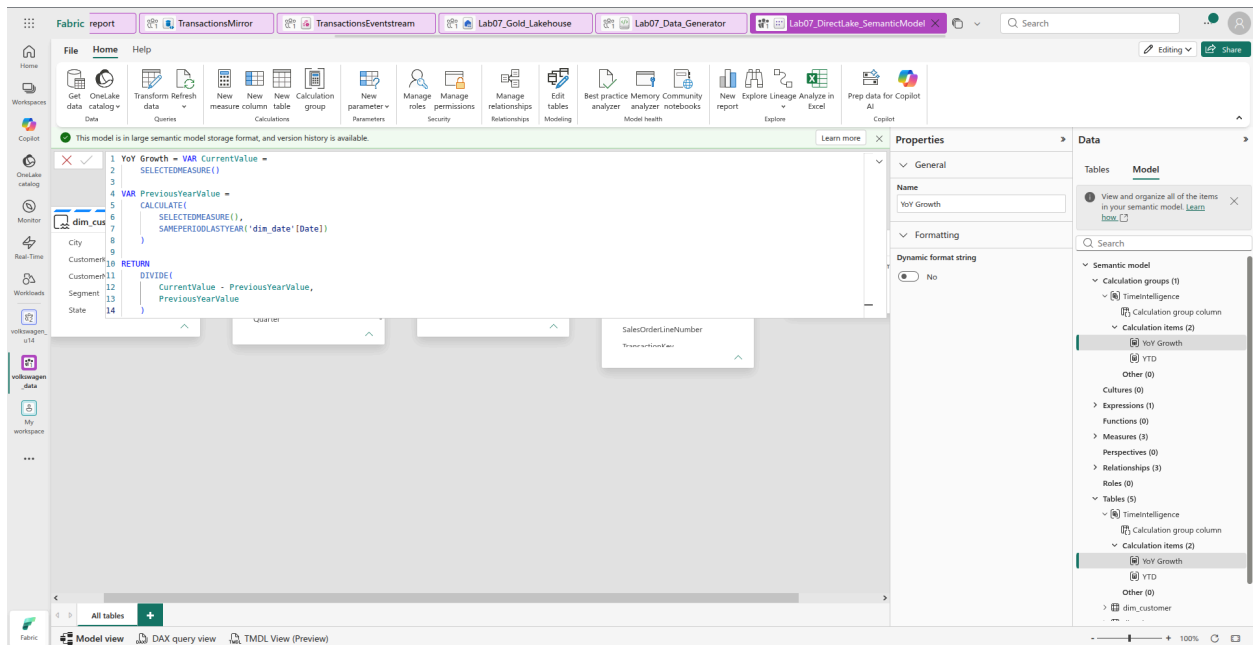
└─ Calculation group column

└─ Calculation items (2)

└─ YTD

└─ YoY Growth

This matches the lab's requirement for a TimeIntelligence calculation group containing **YTD** and **YoY Growth** items.



STEP 05 — Create the display folders

We will organize the measures like this:

fact_sales

|

└─ Finance Metrics

|

└─ Total Revenue

|

└─ Total Cost

|

└─ Total Profit

|

└─ Sales Operations

|

└─ Total Quantity

|

└─ Transaction Count

Part A — Put the existing 3 measures into Finance Metrics

1. Select Total Revenue

In the right **Data** → **Tables** pane:

fact_sales → **Total Revenue**

Click Total Revenue.

2. Find Properties

On the middle/right **Properties** panel, look for:

Display folder

It should currently be empty.

Enter:

Finance Metrics

Press **Enter**.

3. Do the same for Total Cost

Select:

fact_sales → **Total Cost**

Set:

Display folder = Finance Metrics

4. Do the same for Total Profit

Select:

fact_sales → **Total Profit**

Set:

Display folder = Finance Metrics

Part B — Create Sales Operations measures

We currently don't have an operational measure, so we'll create two.

5. Create Total Quantity

Click **New measure**.

Enter **only this**:

Total Quantity = SUM(fact_sales[Quantity])

Click ✓.

Then select the new Total Quantity measure.

In **Properties** → **Display folder**, enter:

Sales Operations

6. Create Transaction Count

Click **New measure** again.

Enter:

Transaction Count = DISTINCTCOUNT(fact_sales[TransactionKey])

Click ✓.

Set its:

Display folder = Sales Operations

Part C — Verify

In the model pane, fact_sales should now contain the two folders:

where to go in properties

Yes, you're exactly at the right place. 👍

In your screenshot, the **second calculation item** is selected. On the right under **Properties** → **General**, you can see:

Name

[Calculation item]

Step 1 — Rename it

Click the **Name** box on the right.

Currently:

Calculation item

Change it to:

YoY Growth

Press **Enter**.

Step 2 — Enter the formula

Now look at the **formula bar at the very top**. It currently says:

Calculation item = SELECTEDMEASURE()

Select **everything in the formula bar** and replace it with:

```
CALCULATE(  
    SELECTEDMEASURE(),  
    SAMEPERIODLASTYEAR('dim_date'[Date])  
)
```

Then click the  **green checkmark** at the top-left.

One correction

The formula above calculates **previous-year value**, but the lab calls the item **YoY Growth**. To actually calculate growth percentage, use this instead:

VAR CurrentValue =

```
    SELECTEDMEASURE()
```

VAR PreviousYearValue =

```
    CALCULATE(  
        SELECTEDMEASURE(),  
        SAMEPERIODLASTYEAR('dim_date'[Date])  
    )
```

RETURN

```
    DIVIDE(  
        CurrentValue - PreviousYearValue,  
        PreviousYearValue  
    )
```

Use this second formula for YoY Growth.

After saving, your right panel should show:

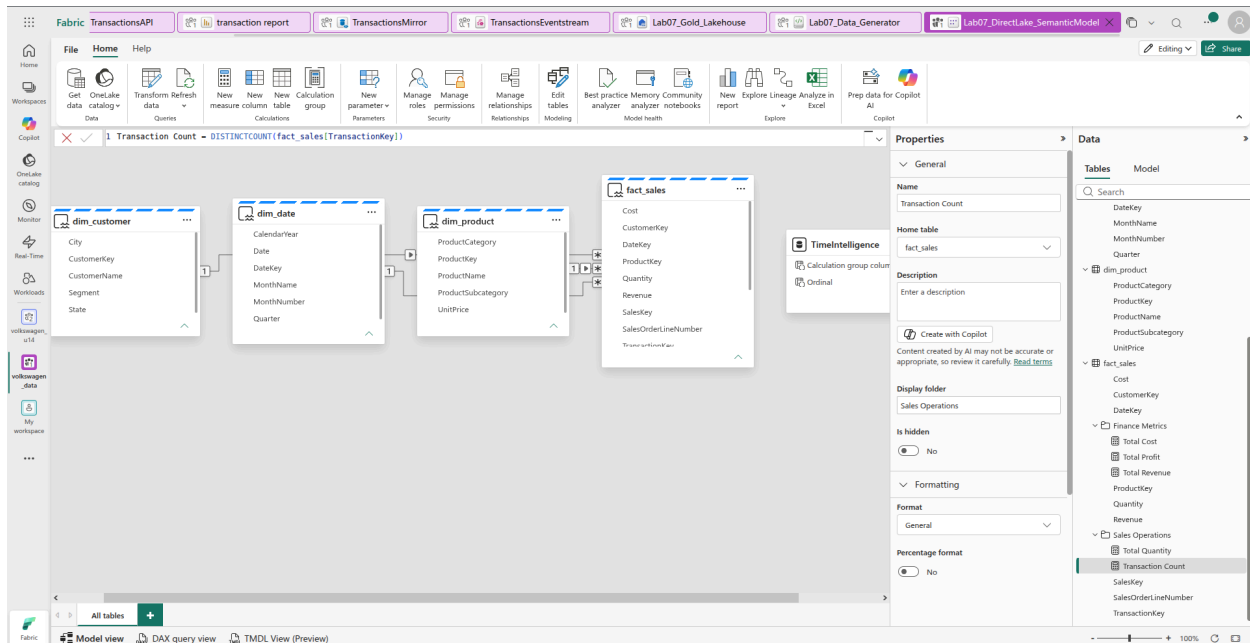
TimeIntelligence

└─ Calculation items (2)

└─ Calculation item ← rename this to YoY Growth

└─ YTD

So first **rename** Calculation item → YoY Growth, then enter the formula above.



STEP 07 — Validate Direct Lake Partition Using TMDL View

Objective:

Validate the Direct Lake configuration of the fact_sales table in the Lab07_DirectLake_SemanticModel.

Procedure

1. Opened the Lab07_DirectLake_SemanticModel in the Microsoft Fabric workspace volkswagen_data.
2. Opened **TMDL View (Preview)** from the bottom navigation of the semantic model.
3. In the **Data** pane on the right, expanded the fact_sales table.
4. Dragged the fact_sales table into the TMDL editor to generate its semantic-model metadata definition.
5. Used **Ctrl + F** in the TMDL editor and searched for:

partition

6. Navigated to the fact_sales partition definition and verified the following configuration:

partition fact_sales = entity

mode: directLake

source

entityName: fact_sales

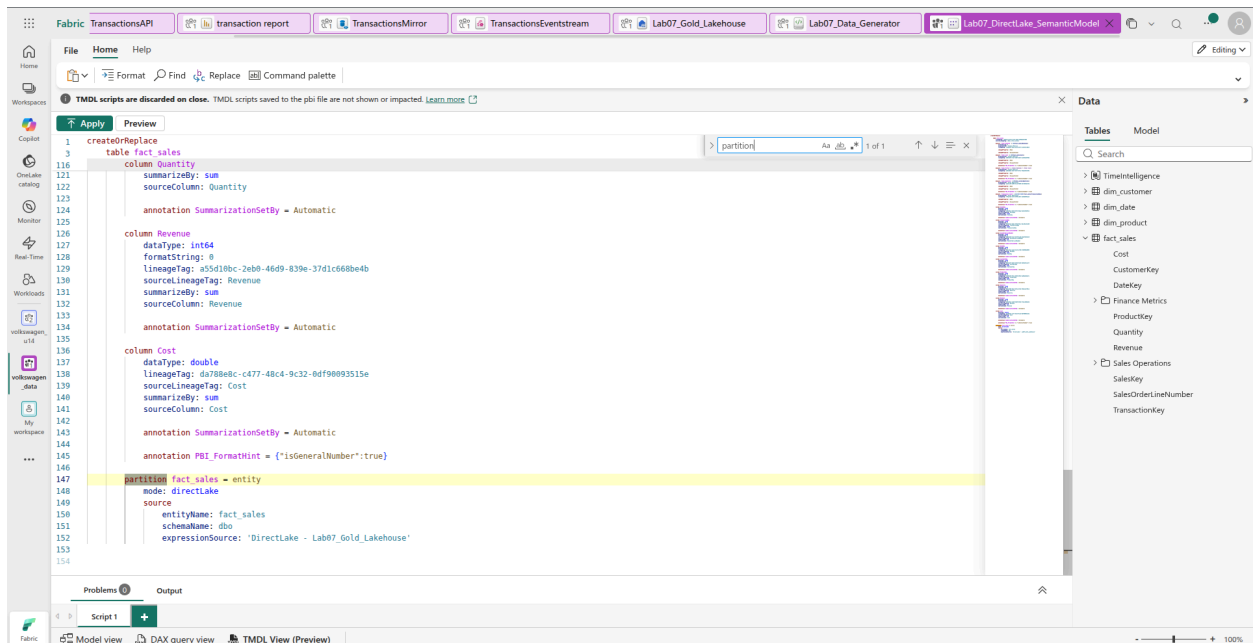
schemaName: dbo

expressionSource: 'DirectLake - Lab07_Gold_Lakehouse'

Validation Results

The TMDL metadata confirmed that:

- The fact_sales table exists in the semantic model. ✓
- A partition named fact_sales exists. ✓
- The partition mode is **Direct Lake**. ✓
- The source entity is fact_sales. ✓
- The schema is dbo. ✓
- The source is the Lab07_Gold_Lakehouse through the Direct Lake expression. ✓
- The TMDL editor showed **Problems: 0**. ✓



STEP 08 — Establish Semantic-Model Version Governance

Step 1 — Opened the semantic model

We were working in:

Lab07_DirectLake_SemanticModel

inside the volkswagen_data workspace.

Step 2 — Opened TMDL View

At the bottom of the semantic model, we selected:

TMDL View (Preview)

Step 3 — Opened the Model pane

On the right side, we selected:

Model

This showed:

- Semantic model
- Calculation groups
- Measures
- Relationships
- Tables

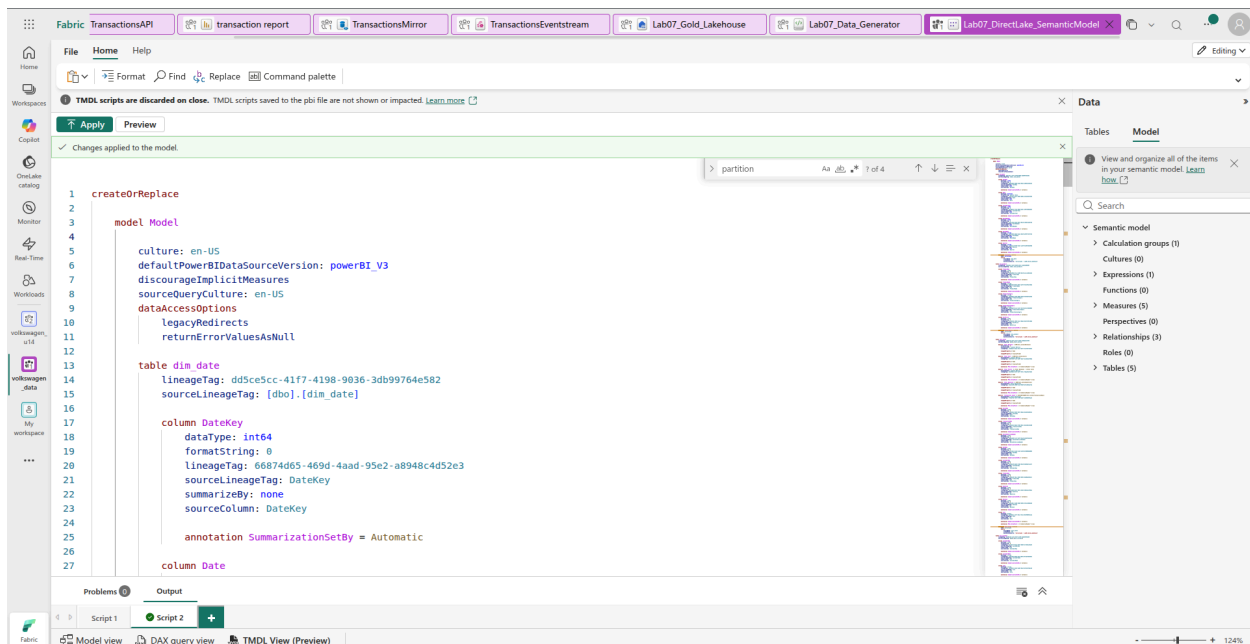
Step 4 — Scripted the model TMDL

We clicked the ... next to **Semantic model**.

Then:

Script TMDL to → Script tab

This generated the complete semantic-model TMDL.



STEP 09 — Capture DAX Performance Baseline

The lab asks us to use DAX Studio and capture performance information for a representative query.

Because you are using **Fabric Web DAX Query View**, we performed the equivalent validation there.

Step 1 — Open the semantic model

Opened:

Lab07_DirectLake_SemanticModel

in the Fabric workspace.

Step 2 — Open DAX Query View

At the bottom of the semantic model, clicked:

DAX query view

Step 3 — Initially ran the benchmark query

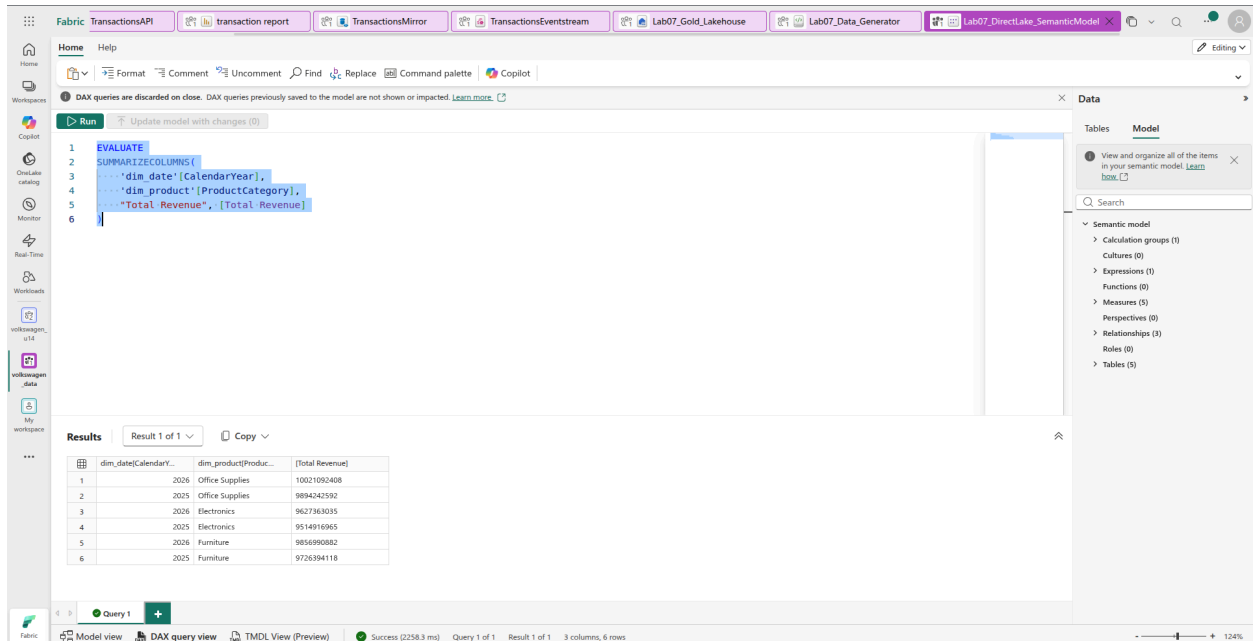
We first used a query containing:

EVALUATE

```
SUMMARIZECOLUMNS(  
  'dim_date'[CalendarYear],  
  'dim_product'[ProductCategory],  
  "Total Revenue", [Total Revenue]  
)
```

Step 6 — Clicked Run

The query successfully executed.



STEP 11 — Inspect Partition Layout

This is what we'll do next.

1. At the bottom of your screen

Click:

TMDL View (Preview)

2. Click the Tables tab on the right

You should see:

- TimeIntelligence
- dim_customer
- dim_date
- dim_product
- fact_sales

3. Expand fact_sales

4. Drag fact_sales into the TMDL editor

5. Press:

Ctrl + F

6. Search:

partition fact_sales

You should find:

```
partition fact_sales = entity
```

```
mode: directLake
```

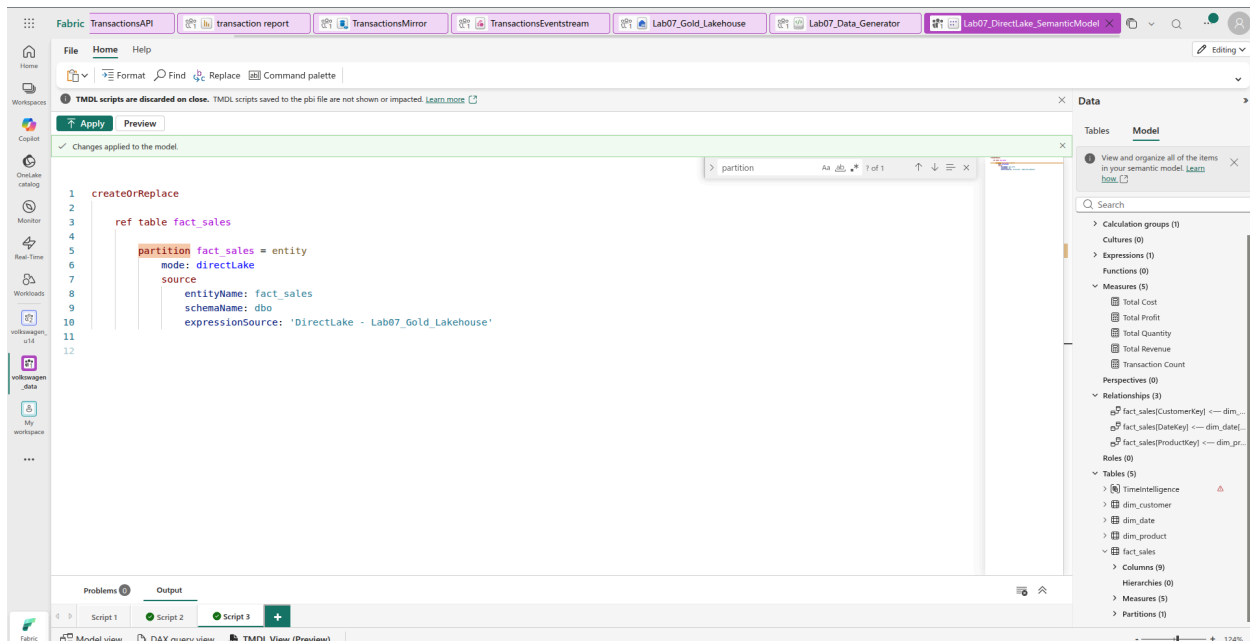
```
source
```

```
    entityName: fact_sales
```

```
    schemaName: dbo
```

```
    expressionSource: 'DirectLake - Lab07_Gold_Lakehouse'
```

This will confirm the **current partition layout**.



STEP 12 — Apply Direct Lake Performance Optimization

According to your lab, **Step 12 has 3 required actions**:

1. Hide SalesOrderLineNumber

In Fact_Sales, hide:

```
Fact_Sales[SalesOrderLineNumber]
```

The reason is that this is a redundant line-item field that isn't needed for user analysis.

2. Split the Fact_Sales partition

Currently your model has:

Fact_Sales

└─ Partitions (1)

└─ fact_sales

So it has **one monolithic partition**.

Step 12 requires changing this to annual partitions:

Fact_Sales

├─ Fact_Sales_2025

└─ Fact_Sales_2026

These should correspond to the Delta/Parquet directories for the two years.

Step 12A — hide the column

1. Stay in your current **TMDL View**.
2. In the right **Model** pane, expand:
fact_sales → **Columns (9)**
3. Find:
SalesOrderLineNumber
4. **Right-click** SalesOrderLineNumber.
5. Choose **Script TMDL to new tab**.

Step 12B — do this next

Add a **new notebook cell** below the failed cell and paste:

```
from pyspark.sql.functions import col
```

```
# Load the existing Fact_Sales table
```

```
fact_sales_existing = spark.table("Fact_Sales")
```

```
# Check the existing data
```

```
print("Rows:", fact_sales_existing.count())
```

```
print("Columns:", fact_sales_existing.columns)
```

Then run this cell

Click the **Run ▶** button for that cell.

Expected result

You should see something like:

Rows: 500000

Columns: ['SalesKey', 'TransactionKey', 'SalesOrderLineNumber',
'DateKey', 'CustomerKey', 'ProductKey',
'Quantity', 'Revenue', 'Cost']

Notebook — Step 12.1: Load existing Fact_Sales

New cell:

```
from pyspark.sql.functions import year, to_date, col
```

```
# Load the existing Fact_Sales table
```

```
fact_sales_existing = spark.table("Fact_Sales")
```

```
# Verify existing data
```

```
print("Rows:", fact_sales_existing.count())
```

```
print("Columns:", fact_sales_existing.columns)
```

Expected output

Rows: 500000

Columns: ['SalesKey', 'TransactionKey', 'SalesOrderLineNumber',
'DateKey', 'CustomerKey', 'ProductKey',
'Quantity', 'Revenue', 'Cost']

Step 12.2: Create the year column

New cell:

```
# Convert DateKey (YYYYMMDD integer) into a date
```

```
# and extract the year for partitioning
```

```
fact_sales_partitioned = fact_sales_existing.withColumn(
  "SalesYear",
  year(
    to_date(
      col("DateKey").cast("string"),
      "yyyyMMdd"
    )
  )
)
```

Verify year distribution

```
fact_sales_partitioned.groupBy("SalesYear") \
  .count() \
  .orderBy("SalesYear") \
  .show()
```

Expected output

```
+-----+-----+
|SalesYear| count|
+-----+-----+
|2025    |250025|
|2026    |249975|
+-----+-----+
```

Step 12.3: Create the physically partitioned Delta table

New cell:

```
# Create a physically partitioned Delta table
# without modifying the original Fact_Sales table
```

```
fact_sales_partitioned.write \
  .format("delta") \
```

```
.mode("overwrite") \  
.partitionBy("SalesYear") \  
.saveAsTable("Fact_Sales_YearPartitioned")  
  
print("Fact_Sales_YearPartitioned created successfully.")
```

Expected output

Fact_Sales_YearPartitioned created successfully.

Step 12.4: Verify Delta partitioning

New cell:

```
spark.sql("""  
DESCRIBE DETAIL Fact_Sales_YearPartitioned  
""").select(  
    "name",  
    "format",  
    "partitionColumns",  
    "numFiles"  
)  
)  
.show(truncate=False)
```

Your actual result

```
format      delta  
partitionColumns  [SalesYear]  
numFiles        2
```

✅ This confirms physical Delta partitioning.

Step 12.5: Verify the annual partitions

New cell:

```
spark.sql("""  
SELECT  
    SalesYear,  
    COUNT(*) AS RowCount  
FROM Fact_Sales_YearPartitioned
```

GROUP BY SalesYear

ORDER BY SalesYear

""").show()

Your actual result

```
+-----+-----+
|SalesYear|RowCount|
+-----+-----+
|2025     |250025  |
|2026     |249975  |
+-----+-----+
```

Step 12.6: Verify no rows were lost

New cell:

```
print(
    "Original Fact_Sales:",
    spark.table("Fact_Sales").count()
)

print(
    "Partitioned Fact_Sales:",
    spark.table("Fact_Sales_YearPartitioned").count()
)
```

Your actual result

Original Fact_Sales: 500000

Partitioned Fact_Sales: 500000

✅ Data integrity confirmed.

Step 12.7: Hide SalesOrderLineNumber

This part was **not done in the notebook**. You did it in the **Fabric semantic model TMDL View**.

The final successful TMDL was:

```
createOrReplace
```

ref table fact_sales

column SalesOrderLineNumber

isHidden

dataType: int64

formatString: 0

lineageTag: 30881b30-4c27-4e79-8ca8-a4ad76e4a418

sourceLineageTag: SalesOrderLineNumber

summarizeBy: sum

sourceColumn: SalesOrderLineNumber

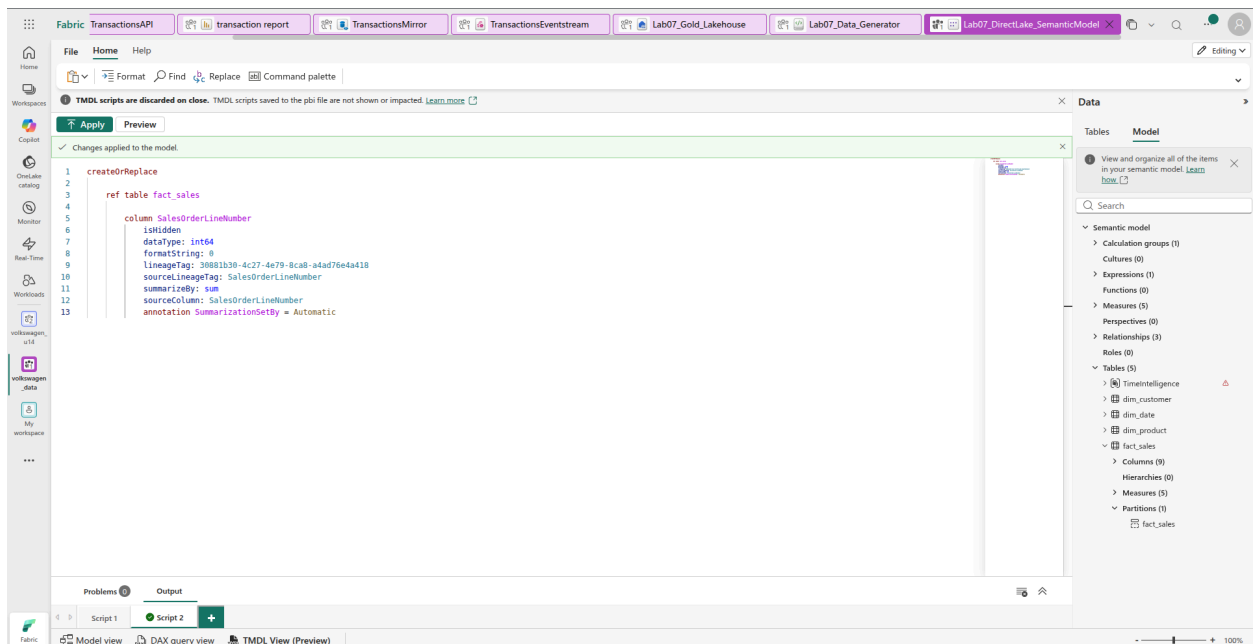
annotation SummarizationSetBy = Automatic

Your screenshot showed:

Changes applied to the model.

Problems: 0

✓ SalesOrderLineNumber successfully hidden.



STEP 13 — Re-run Workload and Compare Performance Metrics.

1. Click your semantic model tab

Lab07_DirectLake_SemanticModel

2. At the bottom click

DAX query view

3. Run this exact same benchmark query you used in Step 09:

EVALUATE

SUMMARIZECOLUMNS(

'dim_date'[CalendarYear],

'dim_product'[ProductCategory],

"Total Revenue", [Total Revenue]

)

4. Run it

Wait for the query to finish.

The screenshot shows the Microsoft Fabric DAX query editor interface. The top bar displays the 'Lab07_DirectLake_SemanticModel' tab. The main editor area contains the following DAX query:

```
1 EVALUATE
2 SUMMARIZECOLUMNS(
3     'dim_date'[CalendarYear],
4     'dim_product'[ProductCategory],
5     "Total Revenue", [Total Revenue]
6 )
```

The 'Results' pane at the bottom shows the output of the query, which is a table with 6 rows and 3 columns: 'dim_date[CalendarYear]', 'dim_product[ProductCategory]', and '[Total Revenue]'. The data is as follows:

	dim_date[CalendarYear]	dim_product[ProductCategory]	[Total Revenue]
1	2026	Office Supplies	10021092408
2	2025	Office Supplies	9894242592
3	2026	Electronics	9627363035
4	2025	Electronics	9514916965
5	2026	Furniture	9856990882
6	2025	Furniture	9726394118

The status bar at the bottom indicates 'Success (2019.8 ms)' and 'Query 1 of 1 Result 1 of 1 3 columns, 6 rows'.

STEP 14 – Validate Consolidated Model Integrity

Step 14.1 – Open the semantic model

You are already in:

Lab07_DirectLake_SemanticModel

At the bottom, click:

Model view

You should see your model with:

- dim_customer
- dim_date
- dim_product
- fact_sales
- TimeIntelligence

Step 14.2 – Open the reporting/exploration view

From the semantic model, open the option to **Explore this data / Create a report** (wording can vary in Fabric).

We need a temporary table visual.

Add these fields:

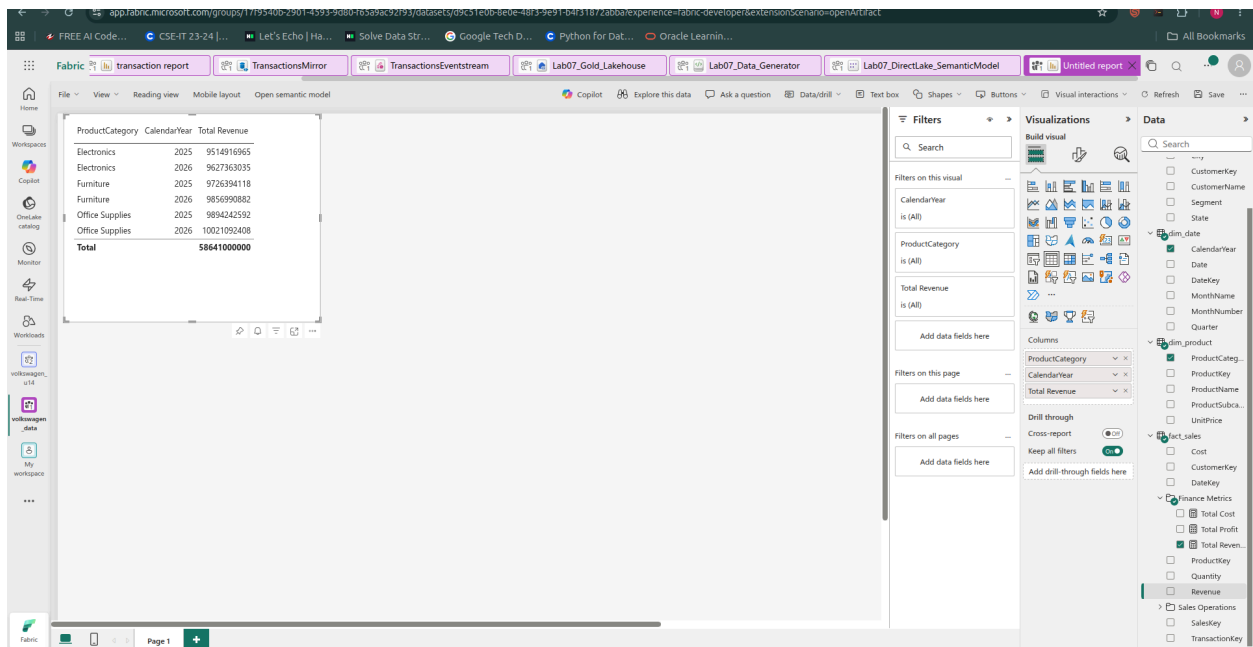
Rows/columns:

- dim_product[ProductCategory]
- dim_date[CalendarYear]

Value:

- [Total Revenue]

Your resulting table should contain combinations similar to:



Step 14.3 – Test the calculation group

Now test the TimeIntelligence calculation group.

Use:

TimeIntelligence → **calculation group column**

and test:

1. **YTD**
2. **YoY Growth**

if not work then add this code in tdm:

createOrReplace

ref table TimeIntelligence

calculationGroup

```
calculationItem YTD =
CALCULATE(
SELECTEDMEASURE(),
DATESYTD('dim_date'[Date])
)
```

```

calculationItem 'YoY Growth' =
VAR CurrentValue =
SELECTEDMEASURE()
VAR PreviousYearValue =
CALCULATE(
SELECTEDMEASURE(),
SAMEPERIODLASTYEAR('dim_date'[Date])
)
RETURN
DIVIDE(
CurrentValue - PreviousYearValue,
PreviousYearValue
)

```

The important thing is that the visual continues to render successfully.

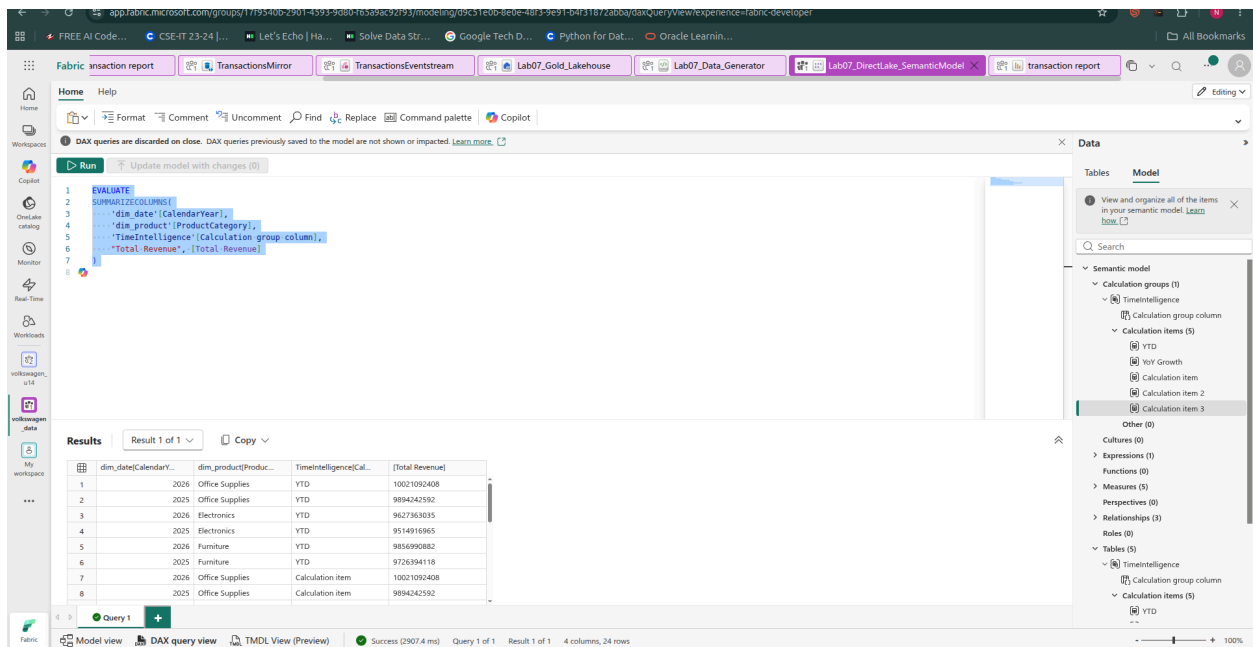
Step 14.4 – Check for errors

go to DAX and run this query:

```

EVALUATE
SUMMARIZECOLUMNS(
'dim_date'[CalendarYear],
'dim_product'[ProductCategory],
'TimeIntelligence'[Calculation group column],
"Total Revenue", [Total Revenue]
)

```



Step 14.5 – Check governance state

Return to the semantic model's **Model view/TMDL View** and verify that your model still contains the governance configuration from Step 08.

Also remember that **Fabric IQ was not exposed in your workspace**, so don't claim that the Fabric IQ mappings were configured.

STEP 14 – Fabric Web Validation Visual.:

app.fabric.microsoft.com/groups/17f9540b-2901-4593-9080-765a9ac92793/datasets/09c31e0b-8e0e-48f3-9e91-b4f31872abba/experience=fabric-developer&extensionsScenario=openArtifact

FREE AI Code...CSE-IT-23-24 [L...Let's Echo | Ha...Solve Data Str...Google Tech D...Python for Dat...Oracle Learnin...

transaction reportTransactionsMirrorTransactionsEventstreamLab07_Gold_LakehouseLab07_Data_GeneratorLab07_DirectLake_SemanticModelUntitled report

FileViewReading viewMobile layoutOpen semantic model

CopilotExplore this dataAsk a questionData/drillText boxShapesButtonsVisual interactionsRefreshSave

HomeWorkspacesCopilotOverview catalogMonitorReal-TimeWorkloadsvoicereport_u14voicereport_dataMy workspace

ProductCategoryCalendarYearTotal Revenue
Electronics20259514916965
Electronics20269627363035
Furniture20259726394118
Furniture20269856990982
Office Supplies20259894242592
Office Supplies202610021092408
Total58641000000

Filters

Search

Filters on this visual

CalendarYear
is (All)

ProductCategory
is (All)

Total Revenue
is (All)

Add data fields here

Filters on this page

ProductCategory
CalendarYear
Total Revenue

Add data fields here

Filters on all pages

Add data fields here

Visualizations

Build visual

Columns

ProductCategory
CalendarYear
Total Revenue

Drill through

Cross-report
Keep all filters

Add drill-through fields here

Data

Search

CustomerKey
CustomerName
Segment
State
CalendarYear
Date
DateKey
MonthName
MonthNumber
Quarter
dim_product
ProductCategory
ProductKey
ProductName
ProductSubca...
UnitPrice
dim_sales
Cost
CustomerKey
DateKey
Finance Metrics
Total Cost
Total Profit
Total Reven...
ProductKey
Quantity
Revenue
Sales Operations
SalesKey
TransactionKey

FabricPage 1